	Department of Computer Science **Faculty of Computing** **UNIVERSITI TEKNOLOGI MALAYSIA**

SUBJECT NAME:	**COMPUTER ORGANIZATION AND ARCHITECTURE**
SUBJECT CODE:	**SECR 1033**
SEMESTER:	**2 – 2023/2024**
LAB TITLE:	**Lab 2: Arithmetic Equations & Operations**
STUDENT INFO :	Execute the lab in group of two. **Student 1**: *No. 1, 3, 5* / **Student 2**: *No 2, 4, 6* **Name 1:** Gui Kah Sin **Metric No:** A23CS0080 **Link for Video Demo:** ____ **Name 2:** Sabrina Heng Wei Qi **Metric No:** A23CS0265 **Link for Video Demo:** https://youtu.be/et3QlUtd8Vc
SUBMISSION DATE & ITEMS:	**Duration of submission :-** 2 weeks **Submission items in elearning:-** 1. Lab 2 exercise sheet/file (in .pdf), with the links for demo video 5 - 10min, on the cover page. 2. The assembly programs (in .asm).

MARKS: ______

Arithmetic Equation Coding in Assembly Language

Q1. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 1 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

```
INCLUDE Irvine32.inc
.data
var1 word 1
var2 word 9

.code
main PROC
        mov ax, var1     ; LINE1
        mov bx, var2     ; LINE2
        xchg ax, bx      ; LINE3
        mov var1, ax     ; LINE4
        mov var2, bx     ; LINE5
        call DumpRegs
        exit
main ENDP
END main
```

Answer Q1

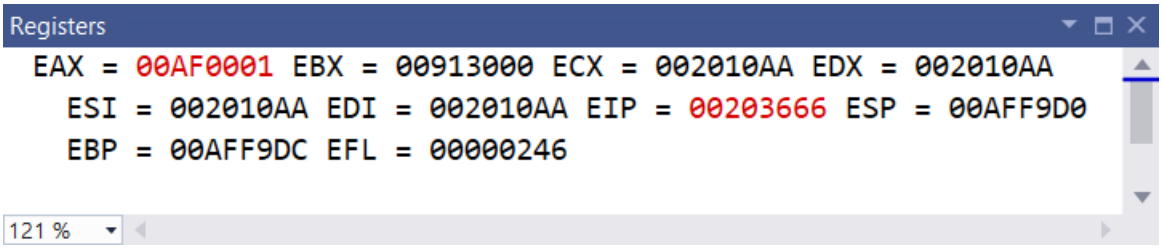
a) Fill (Write) in the contents for the related register in each line:

Table 1

LINE1	**AX = 0001h** **var1 = 0001h**	Move the value of var1 (1d) into register AX
LINE2	**BX =** 0009h **var2 =** 00009h	Move the value of var2 (9d) into register BX
LINE3	**AX =** 0009h **BX =** 0001h	Exchange the value of register AX and BX
LINE4	**AX =** 0009h **var1 =** 0009h	Move the value of register AX into variable var1
LINE5	**BX =** 0001h **var2 =** 0001h	Move the value of register BX into variable var2

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:



LINE2:

Registers

```
EAX = 001E0001 EBX = 00310009 ECX = 008010AA EDX = 008010AA
ESI = 008010AA EDI = 008010AA EIP = 0080366D ESP = 001EF7FC
EBP = 001EF808 EFL = 00000246
```

146 %

LINE3:

Registers

```
EAX = 00AF0009 EBX = 00910001 ECX = 002010AA EDX = 002010AA
ESI = 002010AA EDI = 002010AA EIP = 0020366F ESP = 00AFF9D0
EBP = 00AFF9DC EFL = 00000246
```

121 %

LINE4:

Registers

```
EAX = 00AF0009 EBX = 00910001 ECX = 002010AA EDX = 002010AA
ESI = 002010AA EDI = 002010AA EIP = 00203675 ESP = 00AFF9D0
EBP = 00AFF9DC EFL = 00000246
```

121 %

Autos

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
ax	0x0009	unsigned short
bx	0x0001	unsigned short
var1	0x0009	unsigned short
var2	0x0009	unsigned short

LINE5:

Registers

```
EAX = 00AF0009 EBX = 00910001 ECX = 002010AA EDX = 002010AA
ESI = 002010AA EDI = 002010AA EIP = 0020367C ESP = 00AFF9D0
EBP = 00AFF9DC EFL = 00000246
```

121 %

Autos

Search (Ctrl+E) Search Depth: 3

Name	Value	Type
ax	0x0009	unsigned short
bx	0x0001	unsigned short
var1	0x0009	unsigned short
var2	0x0001	unsigned short

Q2. Execute the program below. Determine output of the program by inspecting the content of the related registers and watches.

a) Fill in Table 2 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: Rval = (-Xval + (Yval – Zval)) + 1

```
include irvine32.inc

.data
Rval DWORD ?
Xval DWORD 26
Yval DWORD 30
Zval DWORD 40

.code
main proc
      mov eax,Xval       ; LINE1
      neg eax            ; LINE2
      mov ebx,Yval       ; LINE3
      sub ebx,Zval       ; LINE4
      add eax,ebx        ; LINE5
      inc eax            ; LINE6
      mov Rval,eax       ; LINE7
      exit
main endp
end main
```

Answer Q2

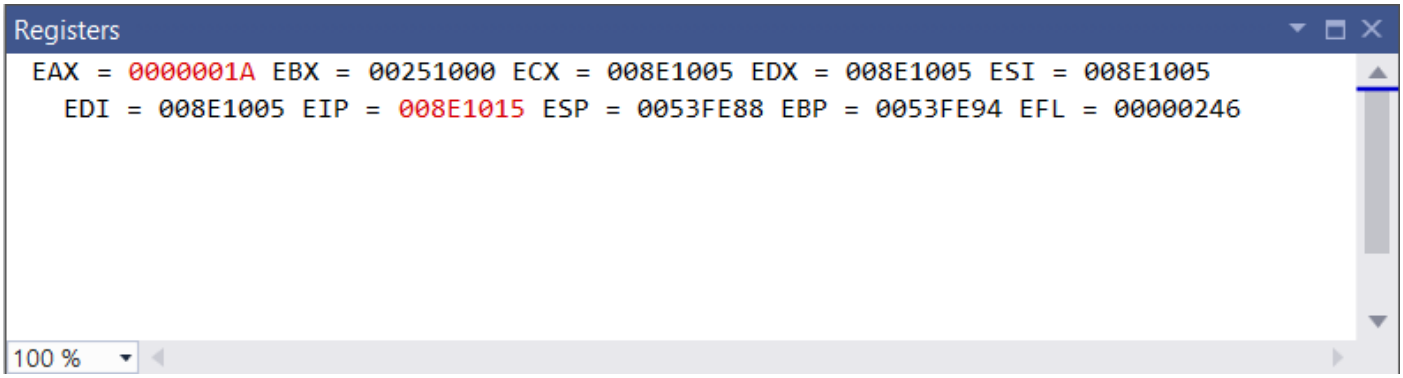
a) Fill (Write) in the contents for the related register in each line:

Table 2

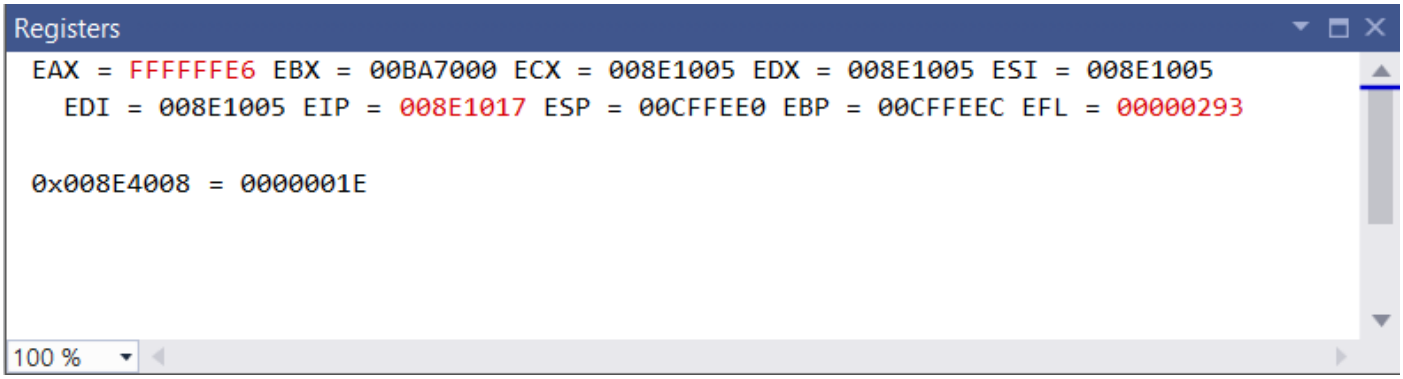
LINE1	**EAX = 0000001Ah** **Xval = 0000001Ah**	Move the value of Xval (26d) into register EAX
LINE2	**EAX =** FFFFFFE6h	Negate the value of register EAX
LINE3	**EBX =** 0000001Eh **Yval =** 0000001Eh	Move the value of Yval (30d) into register EBX
LINE4	**EBX =** FFFFFFF6h **Zval =** 00000028h	Subtract EBX with Zval (40d) and store in register EBX
LINE5	**EAX =** FFFFFFDCh **EBX =** FFFFFFF6h	Add EAX with EBX and store the result in register EAX
LINE6	**EAX =** FFFFFFDDh	Increament of 1 with register EAX
LINE7	**EAX =** FFFFFFDDh **Rval =** FFFFFFDDh	Move the value of EAX into Rval

b) Paste here screenshot of all registers' content after each LINE is executed:

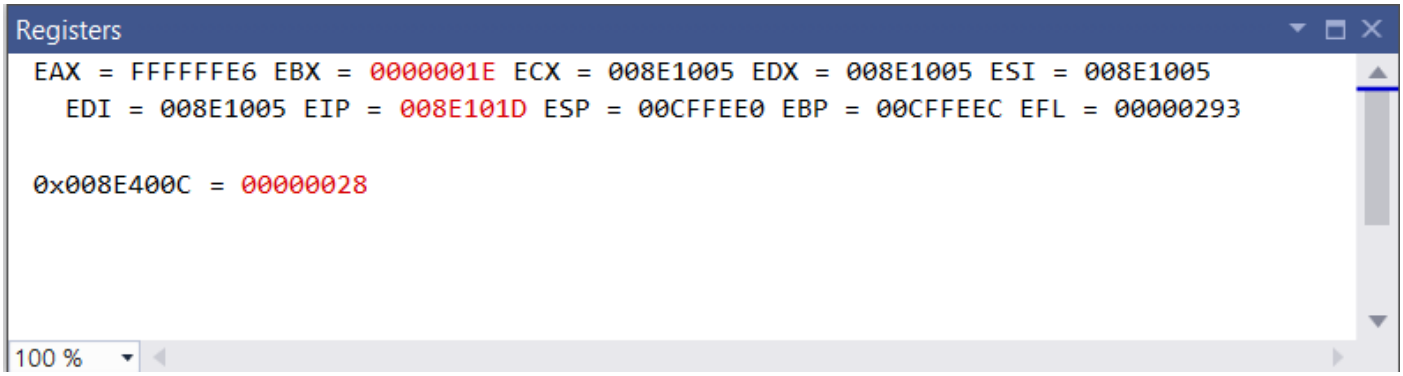
LINE1:



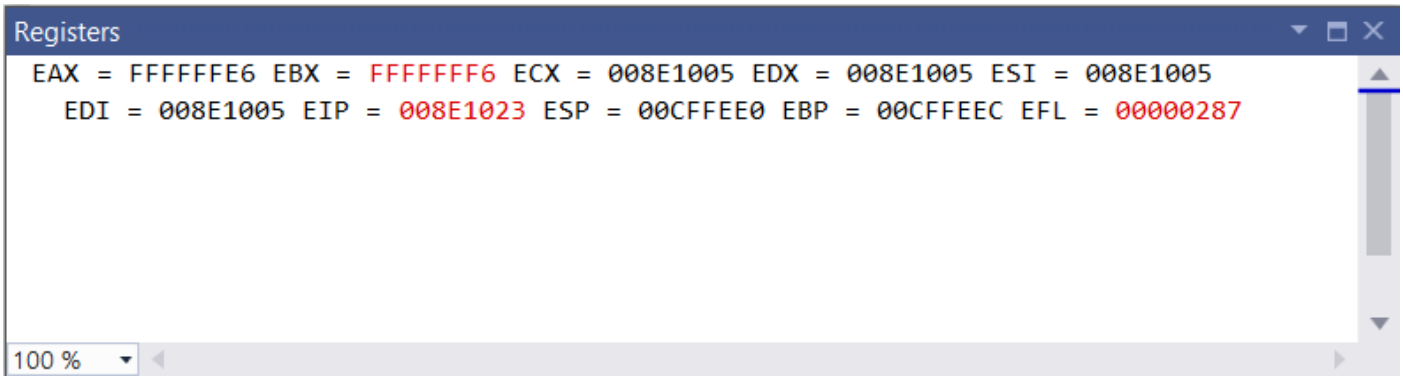
LINE2:



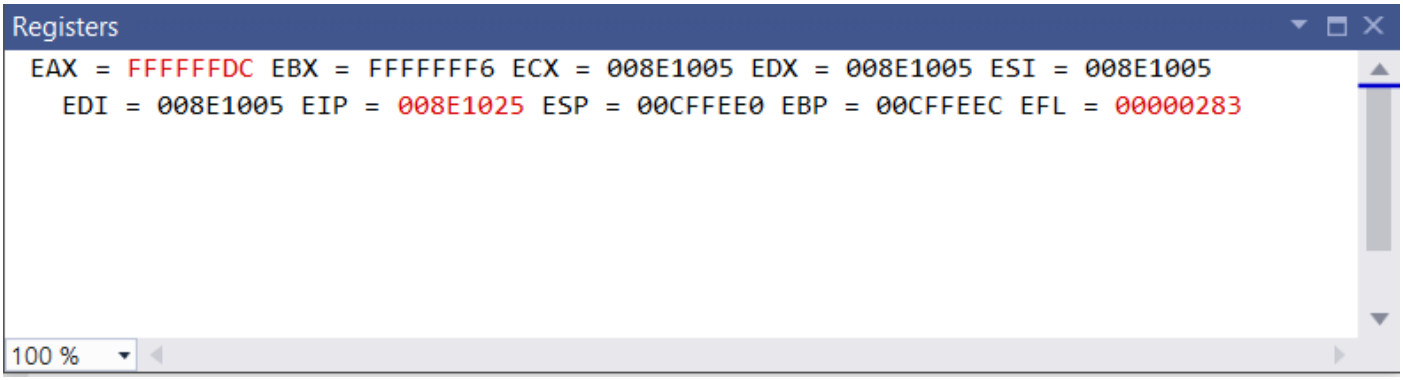
LINE3:



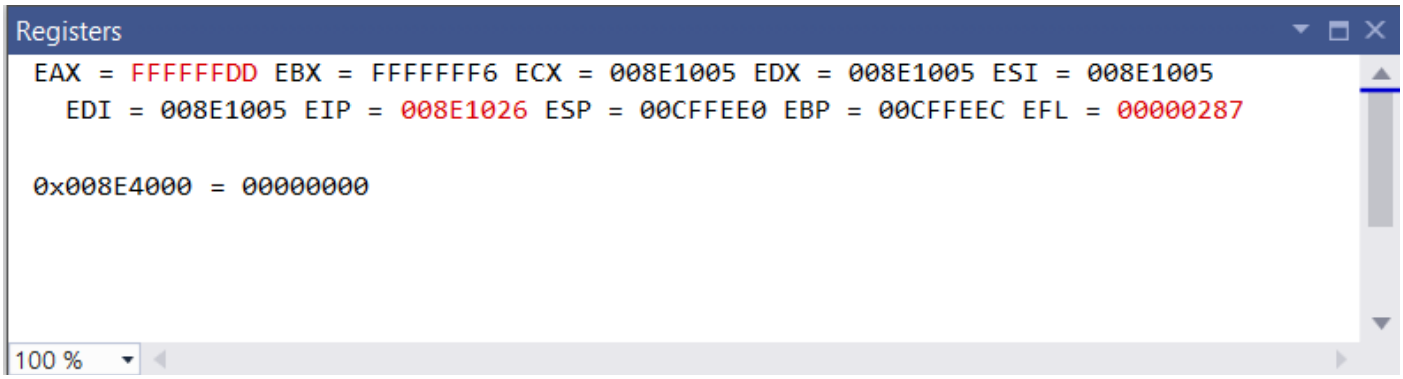
LINE4:



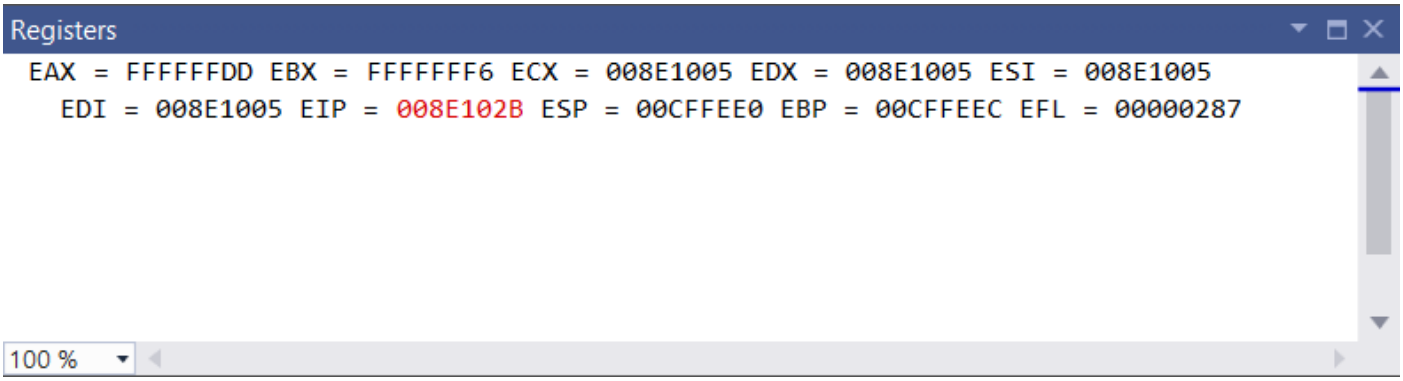
LINE5:



LINE6:



LINE7:



Q3. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 3 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: var4 = [(var1 * var2) + var3] - 1

```
include irvine32.inc

.data
var1 DWORD 5
var2 DWORD 10
var3 DWORD 20
var4 DWORD ?

.code
main proc
      mov eax, var1          ; LINE1
      mul var2               ; LINE2
      add eax, var3          ; LINE3
```

```
      dec eax                ; LINE4
      exit
main endp
end main
```

Answer Q3

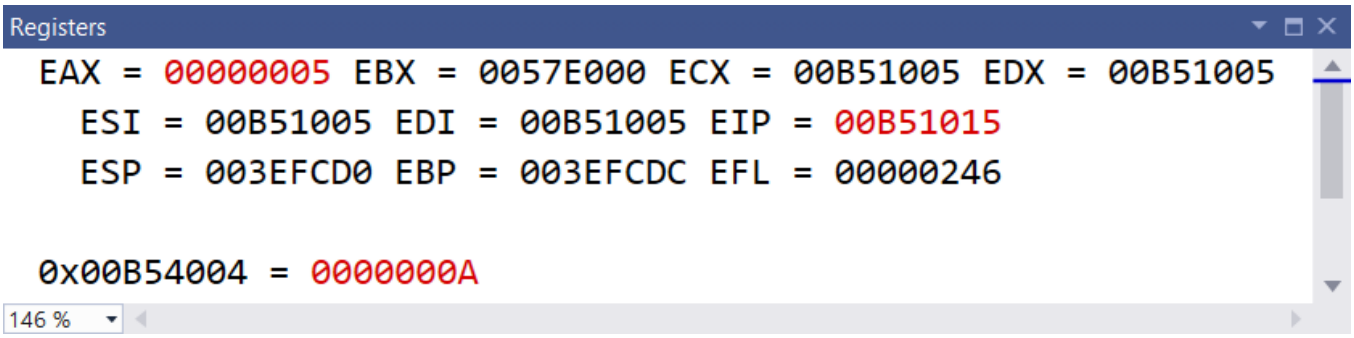
a) Fill (Write) in the contents for the related register in each line:

Table 3

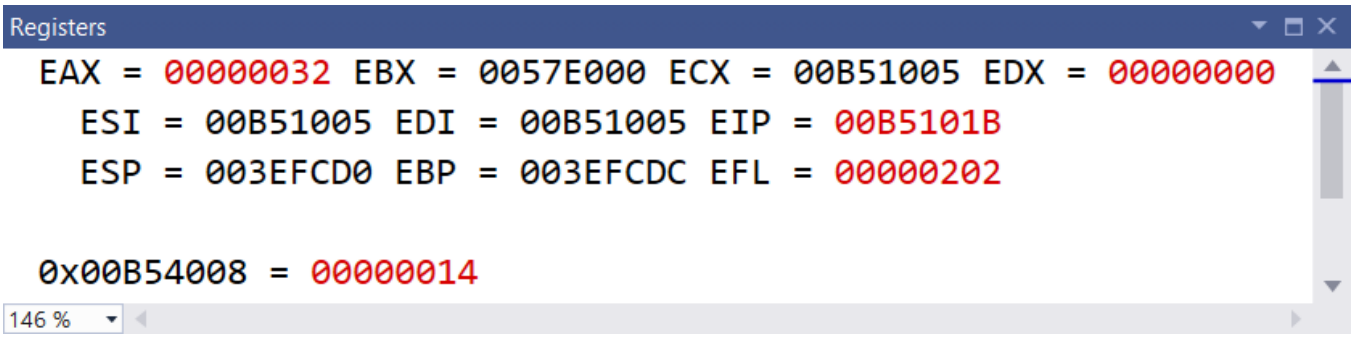
LINE1	**EAX = 00000005h** **var1 = 00000005h**	Move the value of var1 (5d) into register EAX
LINE2	**EAX =** 00000032h **var2 =** 0000000Ah	multiply register EAX by the value of var2 (10d)
LINE3	**EAX =** 00000046h **var3 =** 00000014h	Add register EAX with the value of var3 (20d)
LINE4	**EAX =** 00000045h **var4 =**	Subtract register EAX by 1 and var4 still remain as initial

b) Paste here screenshot of all registers' content after each LINE is executed:

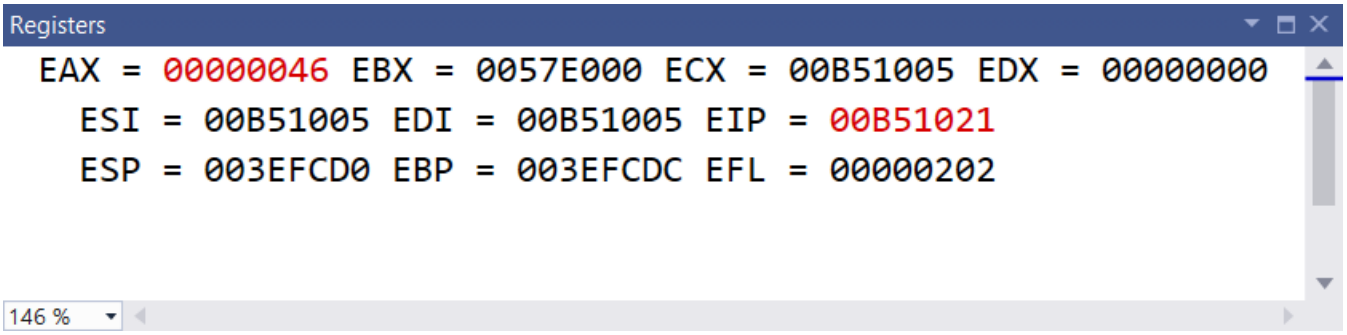
LINE1:



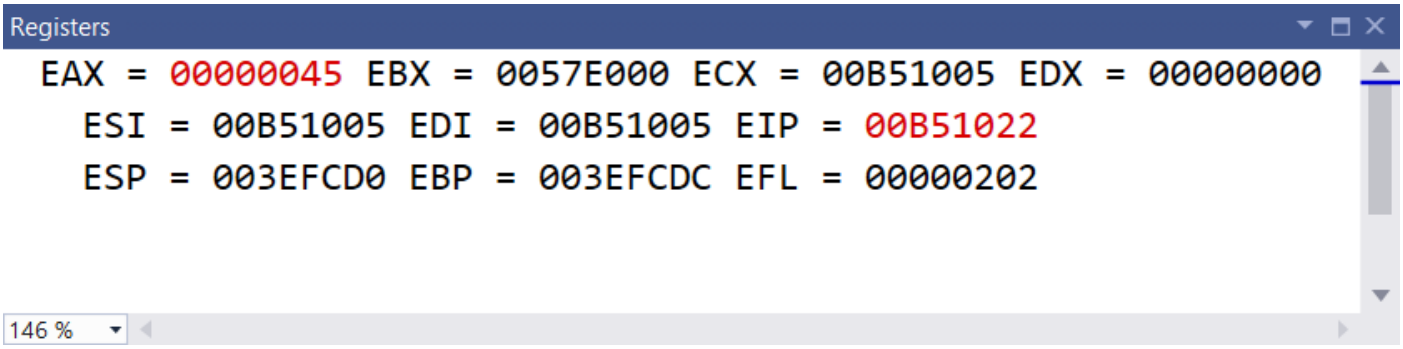
LINE2:



LINE3:



LINE4:



Q4. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 4 with the content of each register or variable on every LINE, in Hexadecimal (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: var4 = (var1 * 5) / (var2 – 3)

```
include irvine32.inc
.data
      var1 WORD 40
      var2 WORD 10
      var4 WORD ?
.code
main proc
      mov ax,var1       ; LINE1
      mov bx,5          ; LINE2
      mul bx            ; LINE3
      mov bx,var2       ; LINE4
      sub bx,3          ; LINE5
      div bx            ; LINE6
      mov var4,ax       ; LINE7
      exit
main endp
end main
```

Answer Q4

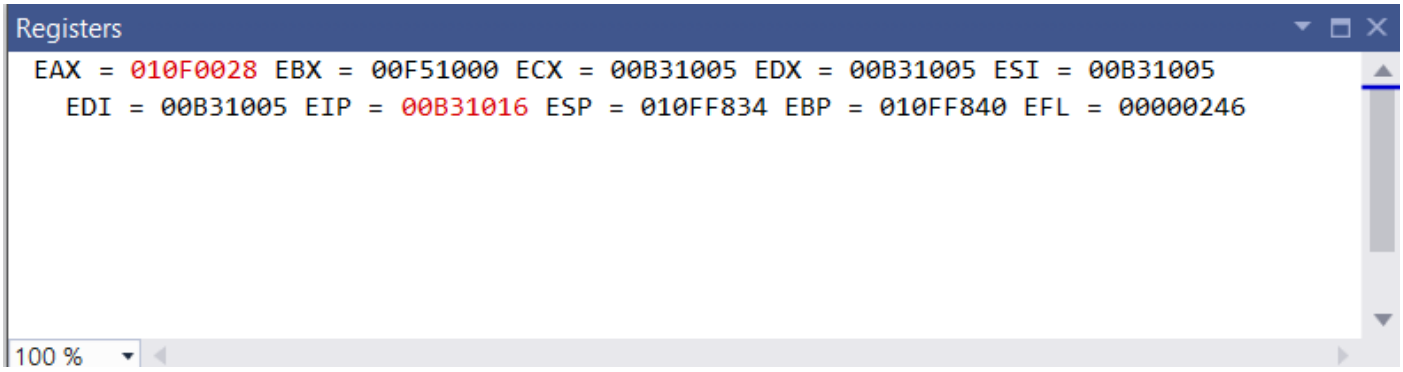
a) Fill (Write) in the contents for the related register in each line:

Table 4

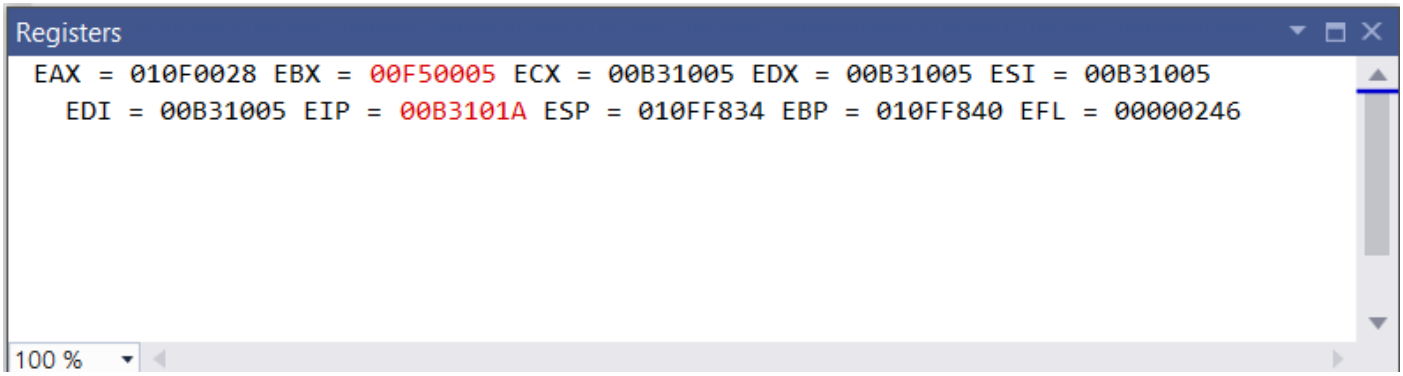
LINE1	**AX = 0028h** **var1 = 0028h**	Move the value of var1 (40d) into register AX
LINE2	**BX =** 0005h	Move the value of 5 into register BX
LINE3	**AX =** 00C8h **BX =** 0005h	multiply AX with BX and store the result in register AX
LINE4	**BX =** 000A h **var2 =** 000A h	Move the value of var2 (10d) into register BX
LINE5	**BX =** 0007 h	Subtract the value of 3 with BX and store in register BX
LINE6	**AX =** 001C h **BX =** 0007 h **DX =** 0004 h	Divide AX with BX and store the quotient in AX but store the remainder in DX
LINE7	**AX =** 001C h **var4 =** 001Ch	Move the value of AX (28d) into the var4

b) Paste here screenshot of all registers' content after each LINE is executed:

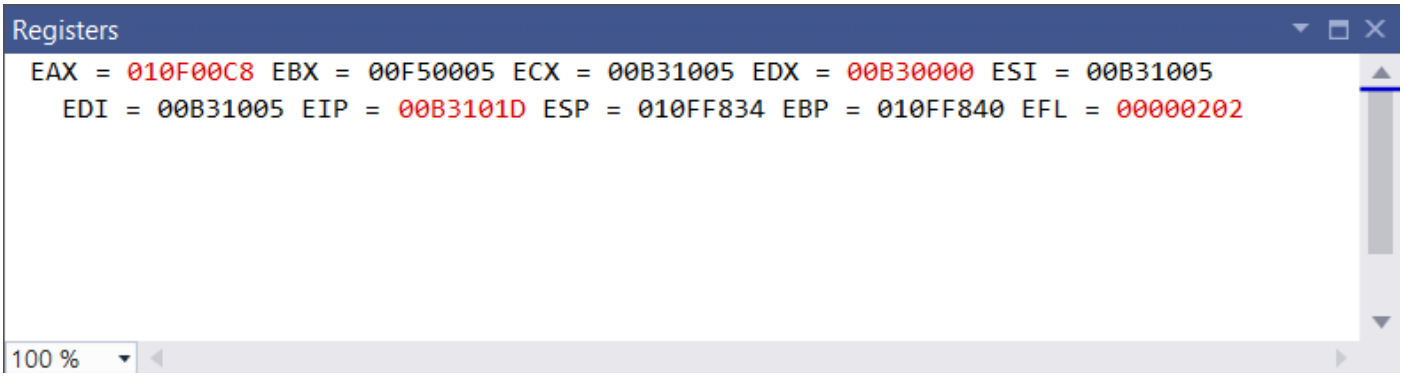
LINE1:



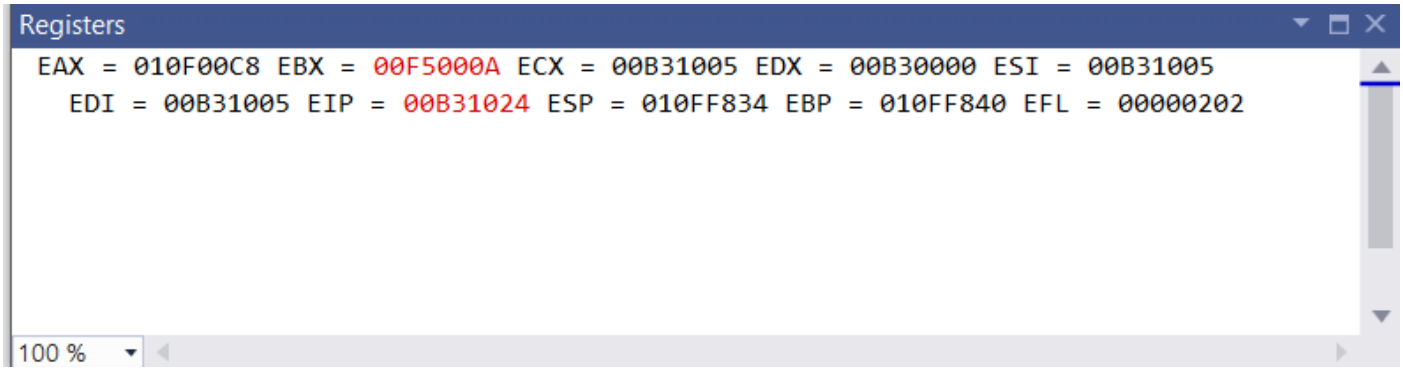
LINE2:



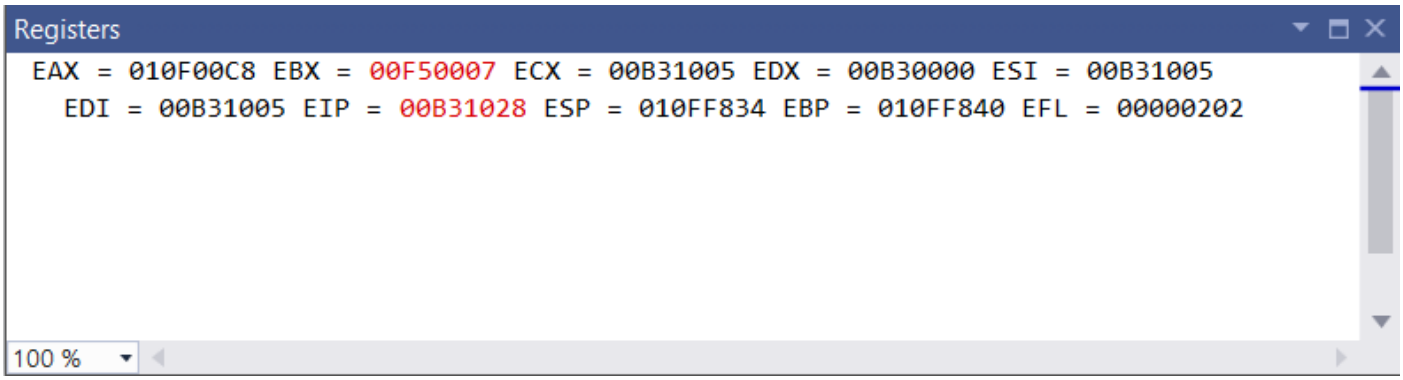
LINE3:



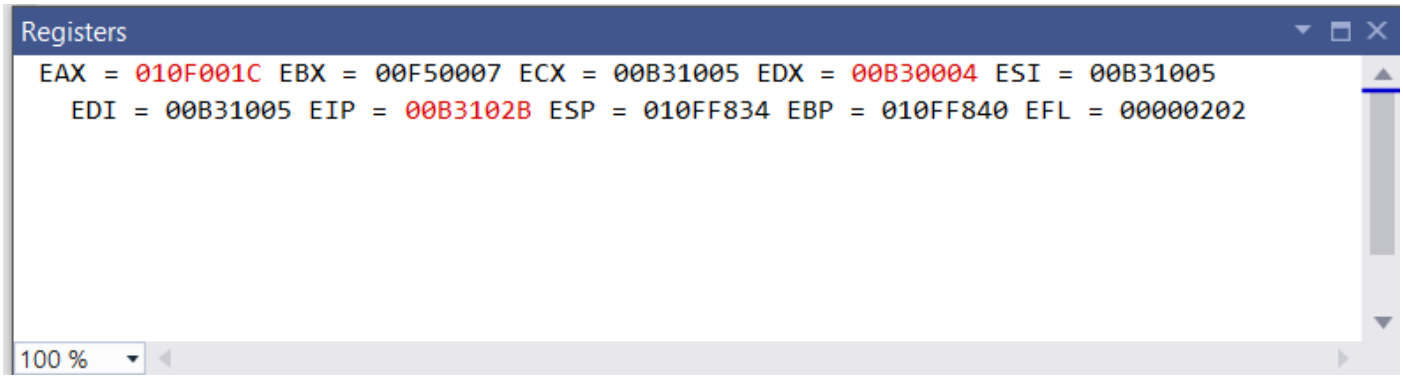
LINE4:



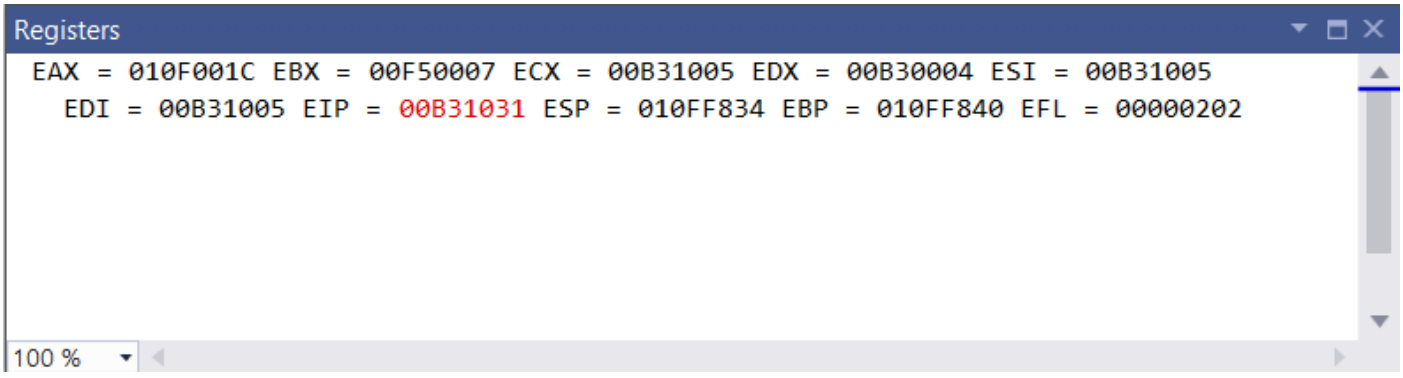
LINE5:



LINE6:



LINE7:



Short Notes for MUL CX and DIV BL:

MUL CX

a. MUL always uses AX (or its extended versions EAX or RAX) as the implicit destination register.

b. The operand size determines the size of the result:

   i. Byte-sized operand: Result in AX

   ii. Word-sized operand: Result in DX:AX

   iii. Doubleword-sized operand (32-bit mode): Result in EDX:EAX

   iv. Quadword-sized operand (64-bit mode): Result in RDX:RAX

c. The upper half of the result (DX or EDX or RDX) holds any overflow bits.

d. The Carry Flag (CF) is set if the upper half of the product is non-zero.

DIV BL

a. DIV always uses the DX:AX or EDX:EAX pair as the implicit dividend register.

b. The divisor is specified as the operand of the DIV instruction.

c. The quotient is stored in AX (for 16-bit division) or EAX (for 32-bit division).

d. The remainder is stored in DX.

e. Clear DX (or EDX for 32-bit division) before division to ensure a correct 16-bit or 32-bit dividend.

f. If the divisor is 0, a division error occurs.

g. The Overflow Flag (OF) is set if the quotient is too large to fit in the destination register.

Q5. Given the following instructions as is Code Snippet 1.

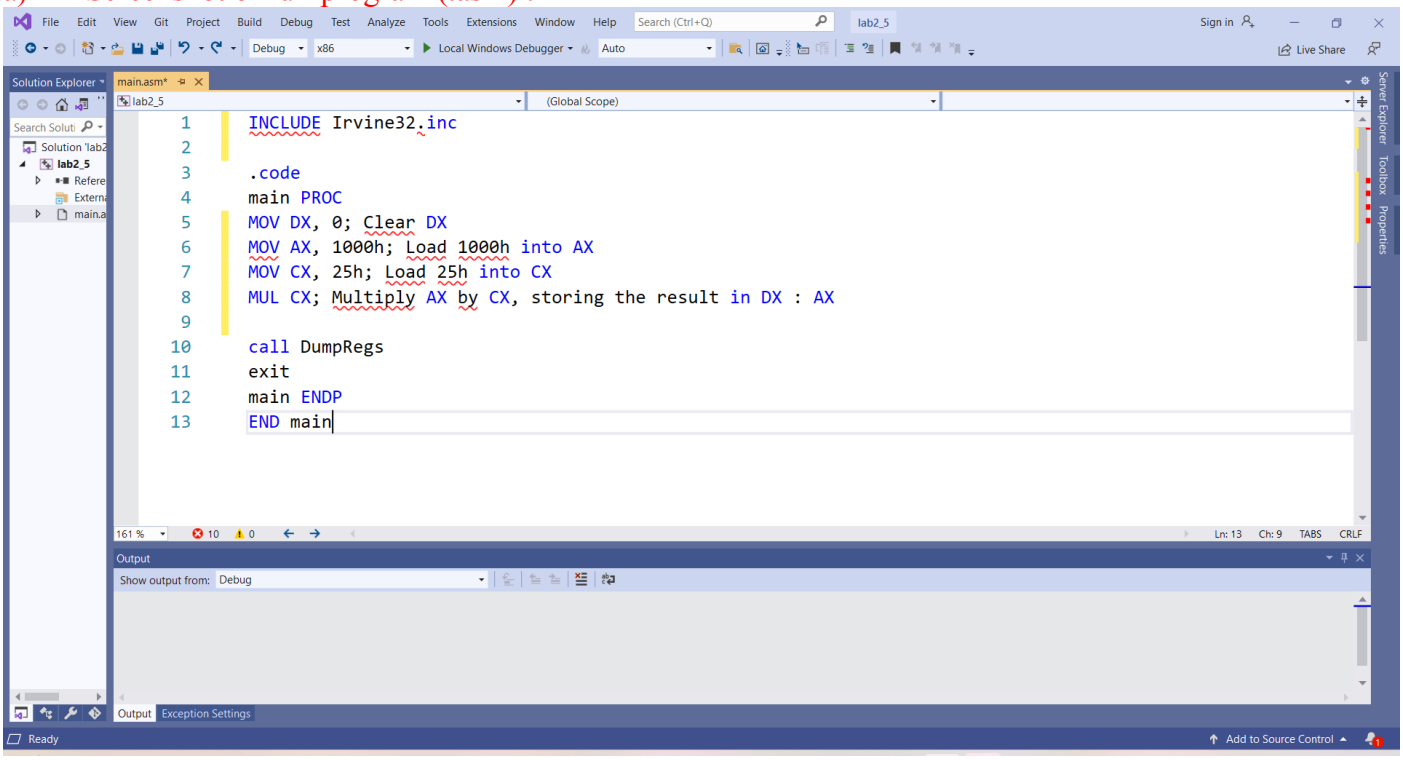
a) Write a full program to execute the Code Snippet 1.
b) What are the contents of the related registers after Code Snippet 1 is executed? Paste the screenshot of DumpReg.

; Code Snippet 1 (MUL CX)

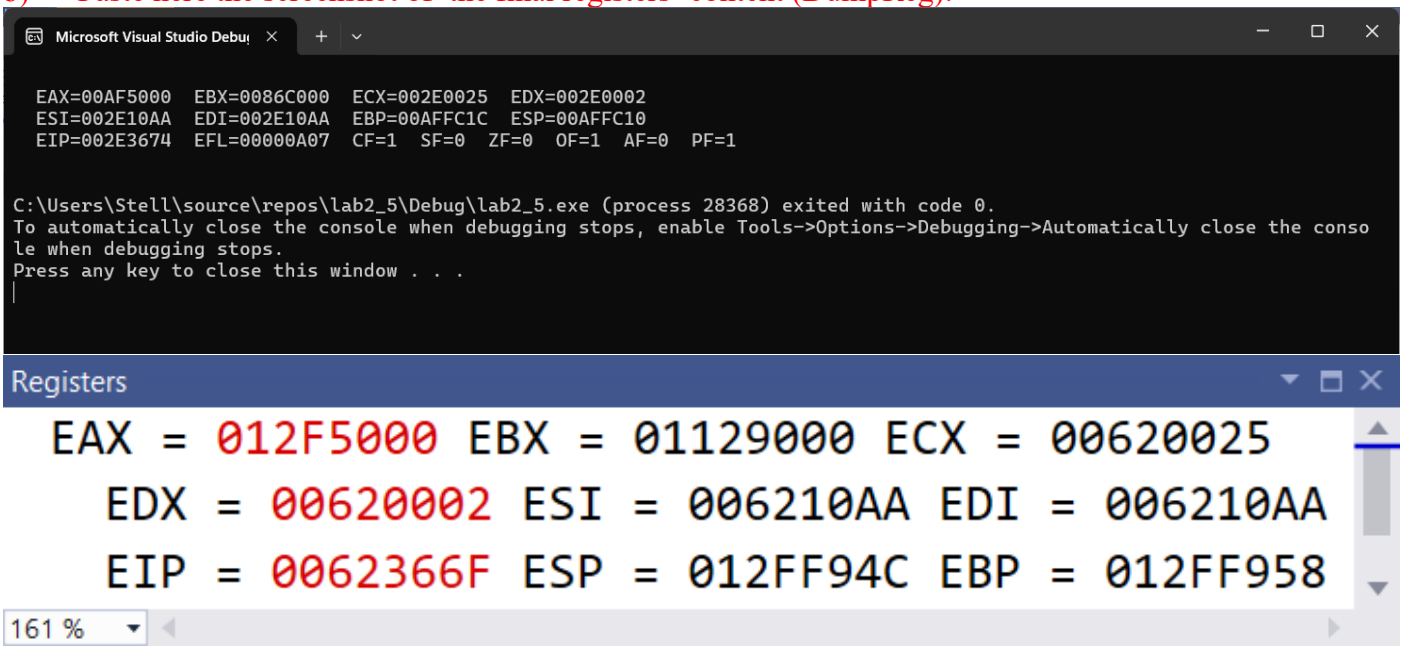
```
MOV DX,0        ; Clear DX
MOV AX,1000h    ; Load 1000h into AX
MOV CX, 25h     ; Load 25h into CX
MUL CX          ; Multiply AX by CX, storing the result in DX:AX
```

Answer Q5

a) Screenshot of full program (.asm) :



b) Paste here the screenshot of the final registers' content (DumpReg):



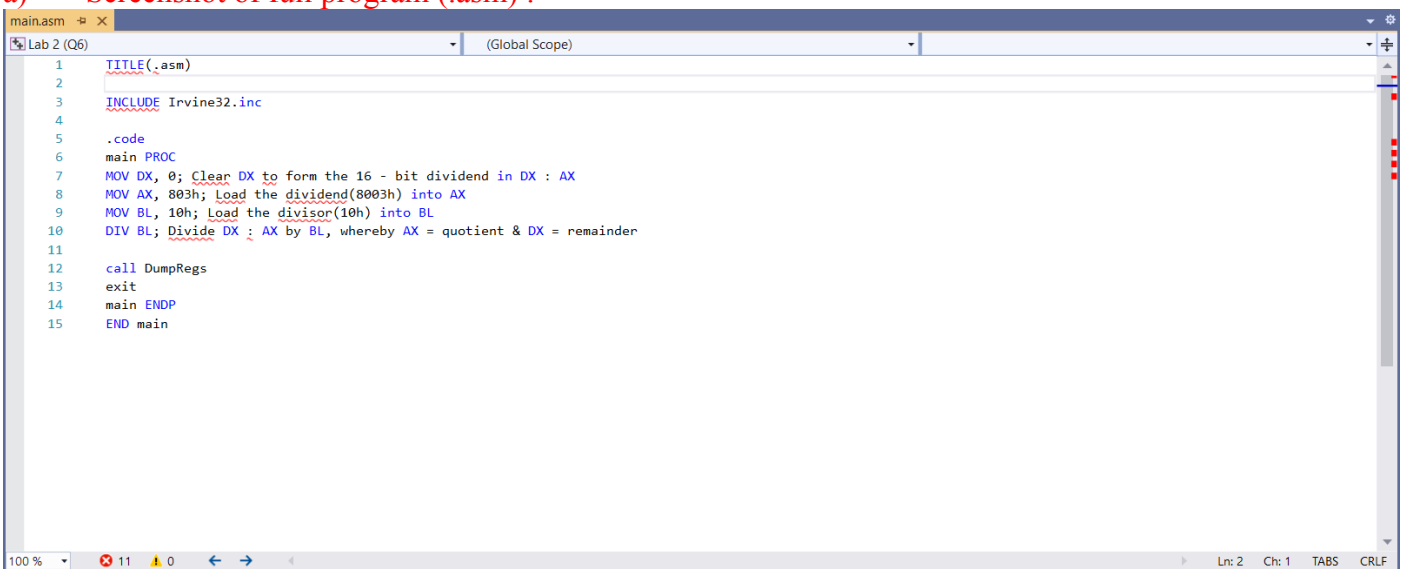
Q6. Given the following instructions as is Code Snippet 2.

a) Write a full program to execute the Code Snippet 2.
b) What are the contents of the related registers after Code Snippet 2 is executed? Paste the screenshot of DumpReg.

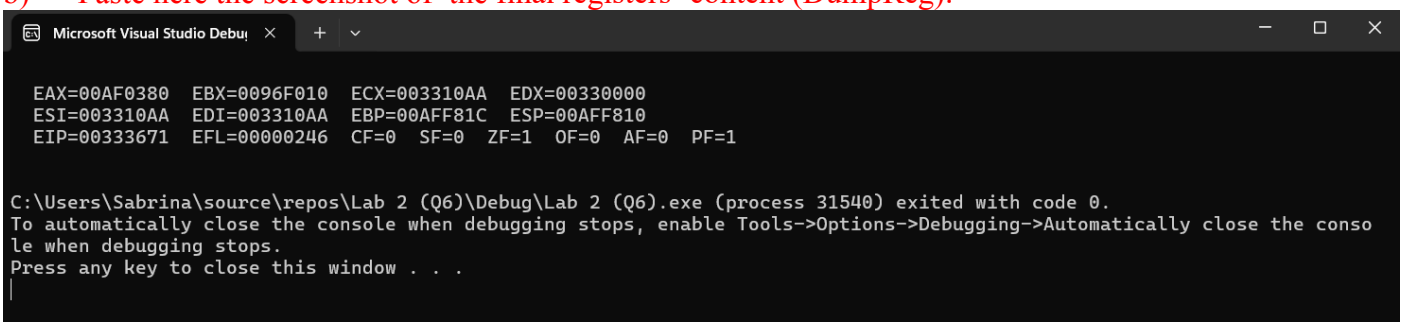
```
; Code Snippet 2 (DIV BL)
MOV DX, 0       ; Clear DX to form the 16-bit dividend in DX:AX
MOV AX, 803h    ; Load the dividend (8003h) into AX
MOV BL, 10h     ; Load the divisor (10h) into BL
DIV BL          ; Divide DX:AX by BL, whereby AX=quotient & DX=remainder
```

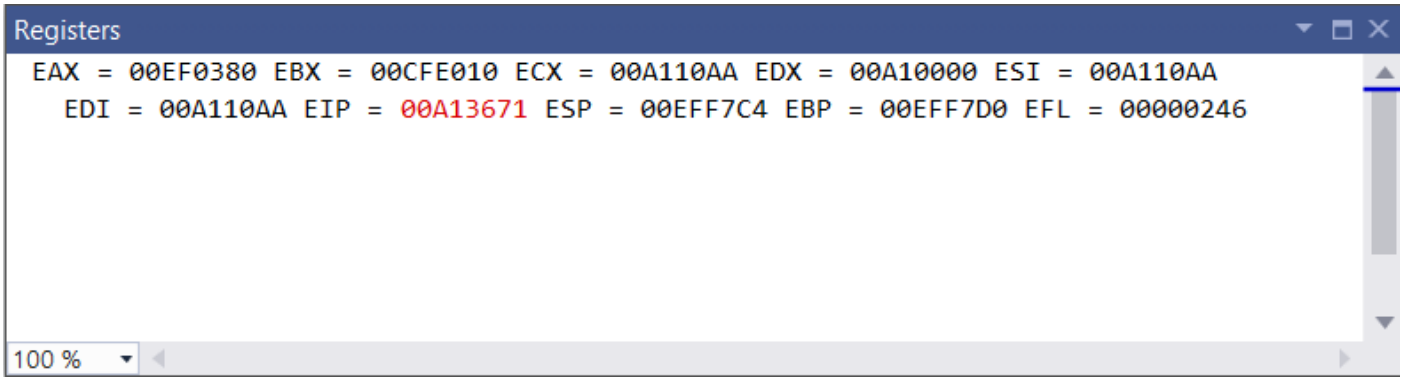
Answer Q6

a) Screenshot of full program (.asm) :



b) Paste here the screenshot of the final registers' content (DumpReg):




Registers
EAX = 00EF0380 EBX = 00CFE010 ECX = 00A110AA EDX = 00A10000 ESI = 00A110AA
EDI = 00A110AA EIP = 00A13671 ESP = 00EFF7C4 EBP = 00EFF7D0 EFL = 00000246
100 %